

Agent 系统设计

1 长会话开始遗漏约束，应该怎样调整上下文和工具加载？

先改供给：查清出错那一步实际收到了什么，把约束、进度和文件版本记在每一步都会带上的任务状态里；改提示词和扩大窗口只作对照。中途新增工具的费用取决于定义加载的位置，不按工具个数翻倍。

到了第 40 轮，模型开始忘记要求、反复读同一份文件。先看出错那一步实际收到的请求，常见的有四种情况，各自指向不同的修改。

请求里看到的情况	原因	该改的地方
约束根本不在请求里	没有保存，或恢复任务时没有读回	保存和恢复流程
约束在压缩后不见了	摘要把它删掉了	压缩规则：关键约束、数字和禁止事项压缩后仍能查回原文
约束还在，模型没有照做	与其他指令冲突、位置不当或任务过难	这时才比较改写提示词、调整位置和扩大 context window
约束已被后来的指令改掉	不属于遗忘	在任务状态里记下变更及生效时间

前两种情况，约束在到达模型之前就丢了，改写提示词补不回来；扩大窗口至多推迟压缩，修不好删掉约束的压缩规则。

1.1 怎样保留任务状态，又避免每轮重读全部材料？

每一步都保留目标、约束和工作进度，文件与工具输出则按需要读取；文件改过以后，只重查依赖旧版本的结论。

接着做一个长任务，既要知道最终要完成什么，也要知道上一步做到了哪里。把这两部分留在当前上下文，模型就不必靠重新翻遍文件来找回进度；至于文件原文，只需在下一步确实要用时补入。

图 1.1 保留目标和进度，按需读材料，再把结果写回进度记录

跨轮保留

- 目标和约束—持续保留
- 当前状态—已完成的工作、采用的文件版本、未解决的问题

按需补入

- 原始材料与工具输出—按下一步需要补入

这一步的上下文 → 下一步所需内容（不必每一步重新携带全部文件和工具输出）

执行 → 读取、调用工具

回写 → 文件变化后，按读取记录判断哪些结论需要重查

进度不能只写“已经检查过合同”，还要记下检查的是哪个版本、得出了什么结论、哪些问题没有解决。文件更新以后，沿这份读取记录就能找到需要重查的结论；任务中断后也能从这里接着做，不会把“曾经读过”当成“现在仍然有效”。

读取记录也把正当的重读和多余的重读分开。文件改了、需要核对原文、上次只读了一部分，都应该再读；同一版本、同一目的、没有带来新信息重读，才是要减少的。权限和客户隔离则不依赖这份记录或模型的记忆，每次调用工具时都重新检查。

1.2 第 40 轮新增三个工具，会让旧内容重新计费吗？

不一定。定义如果改动了已缓存的前缀，受影响的旧内容可能需要重新处理；如果追加在会话后部，旧缓存仍可能命中。

新增三个工具，至少要分清两笔费用：新定义、发现调用和工具执行花了多少，以及原本可以读取缓存的旧内容是否因此多付了钱。第二笔取决于定义放在哪里，不能只根据“第 40 轮新增工具”就认定整段上下文需要重算。

图 1.2 同一批旧内容在三种加载路径下的差额

加载路径	对旧内容的影响	旧内容的差额
修改已缓存前缀	受影响的旧内容重新处理	$150,000 \times (1 - 0.1) \div 1,000,000 = 0.135$ 元
在会话后部追加定义	旧前缀可能继续命中	旧缓存命中时为 0
只更新可搜索目录	完整 schema 可能尚未载入	核对目录摘要和实际加载记录后计算，不能直接记为 0

比较前提：模型和其他配置不变，原缓存仍有效。假设受影响的旧内容为 150,000 token，重新处理为每百万 token 1 元，读取缓存为每百万 token 0.1 元。

旧内容重新处理时，按接口实际适用的普通输入或缓存创建计费，同一批 token 不能两项都算。旧前缀继续命中，只说明这部分没有多付；新定义、发现调用、执行和返回内容仍要另算。

按需发现省下的是每次请求都携带整张工具目录的开销，但发现本身可能多一次调用。几项工具每步都要用，预先加载更简单；目录很大、每次只用其中少数几项，按需发现才更可能划算。运行时已经原生支持 tool search 或 deferred loading 时，直接使用，不必另写一层封装。

“中途加入工具”包含四件不同的事，前一件成立，推不出后一件：

环节	回答的问题	不能据此推出
发现	目录里有哪些相关工具	工具已获准执行
定义载入	schema 何时、在什么位置进入模型上下文	工具服务已经接通

服务接入或热更新	会话不中断时，能否接入、更新或移除工具实现	缓存不受影响，或进行中的调用可以换到新版本
调用授权	现在能否对这个对象执行这个动作	加载或批准过一次，以后都能执行

追加定义不影响旧缓存，不说明撤下或改动已载入的定义也一样：改动位置之后的缓存会失效。客户端收到目录变更通知，也不代表定义已经载入、服务已经接通。沿请求轨迹查看定义在哪一轮、什么位置出现，再用 `usage` 核对缓存命中，才能确认这次走的是哪条路径、表中的估算是否成立。

1.3 怎样证明这些调整改善了长任务？

用同一组长任务做对照，每次只改变工具加载、上下文组织或窗口大小中的一项，再比较结果和总开销。

账单下降，不代表模型不再漏掉约束。按需发现可能多一次调用，选错工具还会带来重试；如果同时重组上下文、扩大窗口，即使任务做得更好，也分不清是哪项改动起了作用。

要验证的改动	保持不变的条件	比较结果
预加载与按需发现	任务、模型、材料、权限、上下文组织和窗口	工具选择与执行结果、重试、总费用和 p95
原上下文与分层组织	同组长任务、模型、权限、工具加载和窗口	约束遗漏、失效文件误用、不必要重读、恢复成功率、成本和 p95
原窗口与扩大窗口	同组长任务、模型、权限、工具加载和上下文组织	相同的任务质量与开销指标

同一组任务在第 20、40、60 轮等多个位置检查，遗漏不一定总出现在第 40 轮。上下文短了，却丢了必要事实，不能算改好了；加载更便宜，却让任务更难恢复，这笔代价也要算进去。保留下来的，应当是既能完成任务、又确实减少总开销的改动。

来源

来源	发布方	类型	支持的内容
Anthropic —Effective context engineering for AI agents	Anthropic	官方工程文章	按需检索与结构化笔记供给
Claude Code —How Claude Code uses prompt caching	Anthropic (Claude Code)	官方文档	缓存按前缀匹配，改动即失效
OpenAI —Tool search	OpenAI	官方文档	按需发现工具追加到末尾
Anthropic —Tool search tool	Anthropic	官方文档	延迟加载不进初始上下文
OpenAI —Prompt caching	OpenAI	官方文档	追加工具可保留旧前缀缓存

Model Context Protocol —The 2026-07-28 Specification	Model Context Protocol	官方工程文章	目录变更通知不等于已接通
OpenAI —Evaluation best practices	OpenAI	官方文档	持续评估捕捉配置变化

2 怎样在能力、等待时间和成本之间选择？

不给整项任务固定一个 reasoning effort。每一步根据难度和出错代价选择，再看多久能给出有用结果、最终完成工作花了多少钱。

让模型多想一会儿，有时能省掉后面的重试和返工，有时只是多花时间。因此，“推理更强”和“调用更贵”不能分开比较，要看到任务结束，多花的时间和费用换来了什么。

2.1 什么时候值得提高 reasoning effort？

难生成、容易返工的步骤值得尝试更高 effort；有确定算法的步骤交给代码。缺少事实时先找材料，多推理不能补出证据。

一个答案容易检查，不代表容易做出来。例如，程序是否通过测试可以很快知道，找到正确的修复却可能需要多轮尝试。如果低 effort 总是失败，随后又要重新检查和修改，直接提高 effort 反而可能更省。

图 2.1 根据这一步的难点，选择代码、reasoning effort 或补充材料

条件	举例	处理方式
有确定的算法或规则	算术、格式校验、明确的权限条件	代码
边界清楚且检查成本可控	抽取和转换	先用较低 effort，再检查输出
生成困难、存在冲突或返工昂贵	多份材料冲突、规划需同时满足多个条件	试用更高 effort，比较收益
缺少关键事实	更长的思考不会产生新的证据	先找材料、询问用户，或请有相关专业能力的人复核

对边界清楚、检查成本可控的步骤，可以先测较低 effort，再比较升级后的正确率、重试次数和总成本。任务进行中也不必坚持最初的选择：抽取信息时发现材料冲突，就重新判断是否需要升级；复杂规划中的机械计算仍然交给代码。选模型同样依据它在相似任务、工具环境和质量要求下的表现；不同厂商的 effort 档位并不对应相同的计算量，只能在同一任务上实测比较。

工具选择还可能在排序之前就出错。如果正确工具根本没有被检索出来，再准确的排序也找不到它。因此，评估要从候选工具是否齐全，一直查到选中的工具能否完成工作，不能只看请求与工具描述有多相似。

会产生外部副作用的步骤另有规则。发送邮件、覆盖文件或修改线上配置之前，effort 开得再高，模型想清楚了也不等于获准执行；是否放行由应用在执行前检查授权。

按步骤选择 effort，也不必在同一条长会话里反复切换模型或 effort。有的实现把两者计入缓存键，切换会让旧内容重新处理；不同阶段可以交给职责固定、上下文较短的调用，再把编排增加的开销一起计入。

2.2 怎样判断用户的等待是否合理？

从请求发出开始计时，重点看多久出现有用结果；首次反馈、完整结果和最终接受也分别记录。

界面一秒内显示“正在处理”，只能说明系统收到了请求。如果随后很久没有可用内容，用户仍然在等。相比模型何时开始输出，更有意义的是它何时给出能用于下一步工作的结果。

图 2.2 四项时间都从同一次请求发出算起

时间终点	记录的事件	对应判断
首次反馈	界面确认收到请求	系统是否及时回应
首次有用结果	出现可用于下一步工作的内容	用户何时开始获得可用结果
完整结果	系统完成一份可审阅的结果	生成和执行何时结束
最终接受	结果达到验收标准并被接受	任务连同审阅、修改共花了多久

四项时间都从同一次请求发出开始计时；首次有用结果是重点。

初版先设以下目标，再按任务类型统计实际延迟与用户行为，决定怎样调整。

初始目标或触发条件	产品行为
请求发出后 1 秒内	确认收到请求
简单任务首次有用结果的 p95 不超过 3 秒	以可用内容出现计时，不能由打字动画代替
预计任务超过 10 秒	显示已经完成的步骤，允许取消、离开和返回

较长任务可以先交出有用的一部分，让后续工作开始。但如果整份结果还要审阅和修改，这些时间也应计入最终接受，不能用早到的第一段内容掩盖后来漫长的返工。

2.3 成本应该算到一次调用，还是一次完成的任务？

把模型、工具、重试、人工审阅和返工的费用加起来，再除以最终被接受的结果数；一份也没被接受时，单独报告失败花了多少。

便宜的一次调用，可能留下更多工作给下一次调用或人工。因此，比较人工完成和 AI 辅助时，要把同一项工作算完，不能只拿模型价格去和人工时间相比。

图 2.3 完成同一项工作，人工与 AI 辅助各花多少钱（假设演算）

方式	构成	合计
人工完成	人工 12 分钟，60 元	60 元

AI 辅助	人工 5 分钟 25 元 + 模型与工具 27 元 费 2 元
未计入	新增错误带来的返工或其他损失 —

假设演算。时间价值按每小时 300 元折算；未计入新增错误带来的返工或损失。

假设演算中少花的 33 元还不是净收益：新增错误带来的返工或损失还要继续扣除。提高 effort 或换模型也按同样的方法判断：减少的错误概率乘以每次错误的损失，超过新增的模型费、等待和验证成本，升级才值得。这是满足安全要求以后的经济判断，省下的钱不能抵消越权。

2.4 不展示原始思考，怎样让工作过程可信？

展示用了哪些材料、改了什么、执行结果是什么，让每项判断和动作都有可核对的依据。

检查一份文件时，可以显示读到的版本、发现的冲突和仍缺的事实。这些信息说明工作为什么停在这里，也让人能够纠正遗漏。准备发送或修改文件时，再显示具体内容、目标对象和影响范围，确认的才是即将发生的动作。

执行后给出回执，并核对结果是否符合预期，才能把“准备做”与“已经做成”区分开。这些记录还可以用于中断恢复和事后审阅；一段听起来流畅的思考过程，不能代替它们。

结果无法确认时，界面照实显示。例如执行进程已经结束、输出也已保存，服务却在发布结果之前中断：恢复后可以展示已有输出，但执行状态仍标为未确认，也不提供重试。可用的按钮与真实状态一致，比一段解释更能让人判断下一步。

来源

来源	发布方	类型	支持的内容
OpenAI —Reasoning models	OpenAI	官方文档	effort 是速度、成本与质量的取舍
jev-harness-lab —Technical evaluation report	Aitejiu	研究代码仓库	候选召回限制排序的上限
Anthropic —Steering thinking and cost	Anthropic	官方文档	节点成本需累加到任务级
OpenAI —Latency optimization	OpenAI	官方文档	减少请求并改用确定性逻辑
Claude Code —How Claude Code uses prompt caching	Anthropic (Claude Code)	官方文档	切换 effort 可能致缓存重算
OpenAI —Evaluation best practices	OpenAI	官方文档	评估须同时看质量延迟成本

3 哪些动作可以自动执行，失败后又怎样恢复？

已获授权的动作可以继续执行；发送、披露或修改数据以后，如果不知道是否成功，就先查结果，不能直接重做。

同样是编辑文件，云端沙箱可能连着生产账号，本地桌面也可能只打开了工作副本。是否需要确认，取决于这次会访问什么、改动什么、把数据发到哪里，而不能只看程序运行在云端还是本地。

3.1 什么时候需要重新取得确认？

账号、对象、内容或后果超出了已有授权，就要在执行前确认；仍在授权范围内的动作无需重复询问。

允许读取一封邮件，不等于允许把它发出去，即使用的是同一个账号和工具。所以要确认的是这一次具体要做什么，而不是笼统地问能不能用某个工具。

图 3.1 按具体动作的后果决定是否需要确认

类别	动作
已在授权范围内，继续执行	读取、整理、草稿
先确认授权覆盖这次具体后果	发送、披露、覆盖关键文件、改变生产状态

批准发送时，要确定收件人、正文版本和有效期限；修改资源时，还要确定改动范围。正文、收件人或目标变了，就重新检查原批准是否仍然适用。网页或邮件要求上传文件，只是读到了一条外部指令，不能把它当作用户已经同意。

确认还必须早于数据真正离开的时刻。敏感内容一旦输入联网表单，就可能已经上传；等到点击提交才询问，已经来不及阻止这次披露。

3.2 Agent 失败后，什么时候可以自动重试？

确认原请求已经终止且未提交，或接口仍能保证同一请求不会重复产生副作用，才可以有限次重试。权限拒绝必须停止。

点击发送后超时，邮件可能已经送出，只是客户端没有收到回执。这时直接重发，就可能让收件人收到两封邮件。因此，unknown 之后先查权威发送记录，再决定下一步。

图 3.2 发送超时后，先查邮件是否已经送出，再决定能否重试

点击发送（授权已覆盖这次后果）

- 取得发送记录或业务回执 → 已发送 → 记录回执
- 请求超时，客户端没有收到结果 → unknown → 查询权威发送记录或业务回执

- 权威记录显示已送出 → **已发送** → 记录回执
 - 原请求已终止且未提交，或仍有有效的幂等性保证 → **可有限次重试** → 沿用原请求标识和参数，并在去重有效期内重试
 - 查不到记录（可能是可见性延迟）→ **仍为 unknown** → 继续核验，不立即重发
- 明确失败 → **失败** → 先辨认原因，见下表

失败原因	下一步	成立条件或原因
读取被限流	在预算内退避后重试	读取没有待核验的写入副作用
参数错误	修正参数后重新执行	确认原请求未执行；不能沿用原参数的幂等保证替代核验
权限拒绝	停止	重复尝试或换一条路径不会使动作获得授权

刚发送的邮件查不到，也可能是记录还没同步，也就是 *eventual consistency*。应等待可见性窗口并继续查询；只有确认原请求已终止且未提交，才能排除它稍后完成的可能。如果靠接口的 *idempotency* 保证重试，就必须沿用同一请求标识和参数，并且仍在去重有效期内。*Message-ID* 能帮助查找同一封邮件，本身不提供去重保证。

参数错误要先修正，但改好参数并不证明上次没有执行，仍需核对。权限拒绝则不能靠多试几次或换路径解决。恢复任务时保留原请求标识、输入版本和核验结果，才能知道是继续同一个请求，还是已经改了内容、需要重新批准。

3.3 模型能力提高后，哪些向导步骤可以删除？

先删去已经不需要的操作指导，每次只删一步；确认风险没有超出容差、用户确实少做了工作，再保留这项简化。

模型即使强了十倍，会操作软件，也不等于知道用户愿意接受哪种业务取舍，或已经获准发送文件。教它怎样点击的步骤可以重测，确认意图和授权的步骤则不能仅凭模型变强就取消。

OSWorld 一类基准的成绩提高，说明操作能力变强，不能直接改变权限边界。可以借用它的任务设计，真实账号、权限和中断恢复仍要另行验收。

测试使用未参与调整的任务，比较危险副作用、选错对象、重复执行和中断恢复，同时统计用户介入。可接受的风险上界和所需样本提前确定，不能看到几次没有事故，就据此取消整套向导。

来源

来源	发布方	类型	支持的内容
OpenAI —Computer use	OpenAI	官方文档	敏感环节应在执行前确认

OpenAI —Guardrails and human review	OpenAI	官方文档	执行前检查对象参数与授权窗口
Anthropic —How we contain Claude across products	Anthropic	官方工程文章	按访问范围与凭证边界判断风险
AWS Builders' Library —Making retries safe with idempotent APIs	Amazon Web Services	官方工程文章	同一请求标识才能安全重试
RFC 5322 —Internet Message Format	IETF	技术标准	Message-ID 不提供去重保证
OSWorld 2.0 —Benchmarking computer-use agents on long-horizon real-world tasks	OSWorld	研究基准	任务设计可借用，权限须另验收
OpenAI —Evaluation best practices	OpenAI	官方文档	生产验收需保留真实任务回归

4 类型受约束的输出，怎样才能用于业务决策？

符合 schema 的答案仍然可能选错。要让系统自动放行，还得检查概率分布、在目标业务上校准，并通过独立验收。

模型只能在几个合法选项里作答，解决的是格式和取值范围的问题；如果选错了那一项，业务判断仍然会错。Jev 的 confidence 又取决于分布形状，不直接等于答对的概率。因此，“输出空间锁死”只保证了类型合法；业务上是否选对、概率是否经过校准、重复调用会不会跨过动作边界、能否抵抗误导性输入，都要分别检查，还要知道漏掉严重问题会付出什么代价。

4.1 严重度打出 1.99，人工闸门设在 2.00，规则应该怎么写？

不按均值写，按严重等级的概率写：严重及以上的概率不高于经业务数据校准的放行上限才自动放行，超过阻断线就阻断，介于两者之间送审。

给四个等级分别记 0、1、2、3 分，构造以下两组概率。两组均值都低于 2，都会被“低于 2 放行”接受，但各等级上的概率很不一样。

图 4.1 同一条“低于 2 放行”规则下的两组四等级风险分布

分布	P(0 无害)	P(1 轻微)	P(2 严重)	P(3 极严重)	均值	P(等级 ≥ 2)	“低于 2 放行”
构造分布 A	0	0.01	0.99	0	1.99	99%	放行
构造分布 B	0.5	0	0	0.5	1.50	50%	放行

两组分布均值都低于 2，都会被“低于 2 放行”接受；但分布 A 中 P(等级 ≥ 2) 达 99%，分布 B 中无害与极严重各占 50%。均值把分布压成一个数，严重尾部随之消失。

分布 A 中，“严重”占 99%，均值却只有 1.99。把 1.99 取整为 2，看似堵住了这个例子，却挡不住分布 B：无害与极严重各占 50%，均值只有 1.5。均值把分布压成一个数，严重尾部随之消失，调整小数位补不回来。

规则因此直接读取完整概率向量，按以下顺序判断：

条件	动作
输入不完整、模型或 rubric 版本 未经验收、输入超出校准范围	送审
P(等级 ≥ 2) 不高于放行上限	自动放行
P(等级 ≥ 2) 高于阻断线	阻断
介于放行上限与阻断线之间	送审

放行上限在独立的业务测试集上校准，使严重漏报率的置信上界不超过业务能承受的预算；阻断线按误拦的代价确定。已经送审的事项，没有新证据时，不能因为重试得到更低的分数就改回放行。

4.2 上线前要补哪一份材料？

一份独立的业务验收与发布批准报告：用自有业务样本和独立标注，检查字段、中间量和最终决定，写明严重漏报、覆盖率、转人工比例和国内网络下的故障表现，再由负责人批准。

厂商题集上的校准数字，只说明模型在那批题上的表现；换成自己的文件和业务要求，要重新取样、重新标注。样本覆盖中文、否定、日期和数字、长材料、选项次序、无关信息和对抗内容，同一客户或同一份合同的片段不能同时出现在调参材料和测试材料里。

图 4.2 发布报告要写清什么，只核对最终标签又会漏掉什么

发布报告要写清	内容
标准答案	由独立标注者给出，意见不一致时另行裁决
测试材料	调参、校准和最终测试各用一批，互不重复
一起报告	严重漏报及其置信区间、自动处理覆盖率、转人工比例和成本
网络与故障	国内网络下的延迟、超时率和降级演练结果
用途与批准	批准的用途与禁用用途、模型和 rubric 版本、负责人

只核对最后答案会漏掉的错误：字段选择（选错）→ 后续计算（随之改变）→ 最终通过标签（恰好没有改变）

只核对最后的通过标签也不够：模型可能取错了字段，却碰巧得到相同标签。如果错误字段还要被后续计算使用，这次“答对”反而会掩盖问题。因此，验收要检查会被继续使用的字段和中间量，不能只报一个最终准确率。

同一个模型用于监控趋势，与用于放行生产动作，验收标准不同。报告批准的是具体用途；只在监控中验证过的评估器，不能直接改作发布闸门。

4.3 同一道题三次结果不同，应该检查什么？

先确认三次收到的输入和评分条件是否相同，再检查差异发生在原始数值、标签还是最终动作上。

输入字节、实际模型版本、rubric、选项顺序或包装层变了，三次调用就不能直接相比。排除这些差异后，还要区分波动的后果：小数稍变但动作不变，与一次放行、一次送审，并不是同样的问题。包装层如果把 1.99 转成整数 2，也要查清业务代码实际读取的是哪个字段。

这些信息需要随重复实验一起保存，不能最后只剩一个“一致率”，更不能一直重试到某次通过。同一条轨迹重复很多次，可以检验稳定性，却没有增加新的业务情境；严重漏报的上界仍要用独立业务测试来估计。

只有输入完整、模型版本已经验收、结果仍在校准范围内，才有依据自动判断。把全部请求交给人工，虽然可能压低自动错误，却没有完成自动处理的工作，所以还要一起报告覆盖率。

4.4 服务不稳定和判断不确定，能用同一种降级吗？

服务连不上，就按故障流程恢复或切换；证据不够判断，就补证或送审。两种情况可以返回同类动作，但不能共用一套触发条件。

网络恢复，只说明又能收到响应，并不说明答案可靠。同样，备用模型能返回分数，也不代表原模型的阈值可以直接套用。决定下一步之前，要先分清现在缺的是服务响应，还是作出判断的依据。

图 4.3 网络故障与判断不确定分别处理

情形	处理步骤
网络故障	超时、有限次重试、熔断 → 切到事先验收过的备用方案（本地模型、规则或人工）
判断不确定	看证据是否足够、风险多高 → 送审（review）或标为 unknown

两条路径统一返回 allow、review、deny、unknown，但触发条件不同，分数与阈值不能直接共用。数据去向在接入时确定，不因故障临时更换供应商或传输区域；高影响动作在校验服务不可用时，只保存草稿或交人工处理。

来源

来源	发布方	类型	支持的内容
TypeSafe —Score	TypeSafe	官方文档	分数为等级概率加权，可落在两级之间
TypeSafe —Confidence	TypeSafe	官方文档	confidence 只反映分布集中度
TypeSafe —Jev 1.13 jaggedness	TypeSafe	官方文档	数字日期对抗内容需专项覆盖
TypeSafe —Self-consistency: choices	TypeSafe	官方文档	重复一致率不等于结果正确
Pydantic AI —TypeSafe (Jev)	Pydantic	官方文档	取整等级需核对原始分数字段
Pydantic AI —models.typesafe API	Pydantic	官方文档	latest 别名随发布变动，需记录实际版本

5 律师的一次修改，怎样才能成为平台规则？

两处修改都先保存，但不直接写入全平台生效的 playbook。删句和加公式分别查清理由和依据，批准后只在限定的客户和法域内使用；下周的德国和越南事项，只用各自已批准的规则。

香港律师替客户 A 修改新加坡就业合同，删掉了一句风险提示，又加上一条补偿公式。律师交付这份合同，完成的是客户 A 的事项；平台是否采纳其中的做法，是另一个决定。如果平台只记下“律师改过这份合同”，下次就无法判断该沿用哪一处修改，要让经验能够复用，还必须知道律师为什么这样改。

5.1 为什么删除一句话和新增一个公式要分开审查？

删去风险提示，不代表以后都不必提示；加上一条公式，也不代表别的合同可以照算。两处修改各有需要核实的理由。

律师删掉一句话，可能是因为原句没有依据，也可能只是因为它不适用于客户 A。新增公式又可能出于不同考虑。即使两处修改同时完成，也不能把接受其中一处的理由用于另一处。

图 5.1 删去风险句与加上公式，各自需要哪些理由和检查

- D1 删除：若条款被挑战，供应商可能面临 50% 规则制裁
- + D2 新增：补偿公式 = 月薪 × 12 × 当地 50% 规则系数

D1

1. 律师已删去风险句（删除理由尚待核实）
2. 查明为什么删：原句错误、缺少依据、不适用于这份合同，还是客户在谈判中作出的取舍
3. 请独立复核者检查：删去这句，不代表以后都不必提示这类风险
4. 批准可复用的做法
5. 仅用于获准的客户和法域

D2

1. 律师已加上公式（计算依据尚待核实）
2. 查明公式出处：从哪里来、计算什么、适用于谁
3. 核对各法域的条件：德国——基数、期间和适用范围；越南——没有已批准的依据时保持待审
4. 批准可复用的做法
5. 仅用于获准的客户和法域

删除句中的“50% 规则制裁”与公式中的“50% 规则系数”，必须各自查清指的是什么。数字相同，不能证明它们来自同一条法律规定；客户愿意接受合同约定，也不能证明其他客户都应该使用它。

德国合同即使恰好算出相同金额，也还要核对算法。相关规定中的 50% 是竞业限制补偿下限，基数涉及最后取得的合同给付，浮动给付另有平均规则，计算期间也须与限制期对应。只算月薪乘 12 再乘 50%，可能碰巧答对一个样例，却漏掉这些条件。换成越南，如果还没有取得并批准当地的依据，就不能仅改国家名称、填上一个系数。

5.2 机制要保留什么，修改才能成为规则？

保留原稿、两处改动各自的理由和依据、作者与委托关系、审查和批准记录，以及父版本和适用范围；做法经批准后才写入有版本的 `playbook`，不能只给一个新版本号。

平台可以先学会以后怎样检查合同：哪些句子要查法源，公式要核对哪些条件，遇到什么情况需要复核。这些做法保存在有版本的 `playbook` 中，后续合同就能使用，不必先训练模型权重。要知道一次更新为什么值得采用，就得保留 D1、D2 各自的原稿、理由、反对意见和审查过程；保存下来的历史受权限约束，也不会自动参与以后的回答。

图 5.2 保留修改理由和批准过程，再限定以后哪些合同可以使用

保留的记录	内容
改了什么	原稿、逐项 diff 和父版本
为什么改	作者身份、执业法域、修改理由、法源与反对意见
谁检查和批准过	测试结果、审批人和生效时间

四类信息分开保存（读取和使用时都核对范围）

- 合同事实
- 客户偏好—可以影响起草选择，不能覆盖强制规则或平台的权限要求
- 法律要求
- 平台核查步骤

能否复用，还取决于资料可以给谁使用。客户 A 的红线不能被拿去回答客户 B 的问题，也不能进入为 B 准备的摘要、记忆或评估材料；最后删掉客户名，并不能消除此前已经用过 A 资料的影响。因此，读取材料和使用规则时都要核对客户及适用范围。

客户偏好、合同事实、法律要求和平台流程也不能混在一起。A 偏好的写法可以用于替 A 起草，但不能覆盖强制规定，也不能改变平台的权限要求。交付了一份按 A 要求修改的合同，并不意味着以后所有客户都要照着写。

5.3 独立评估怎样做？

把原版、仅删除、仅新增和两项都改的版本分别测试；法源交给熟悉当地法律的复核者，验收材料、评分标准和发布权限都不由提出修改的一方掌握。

如果只测修改后的整份合同，即使结果更好，也分不清是删除风险句、新增公式，还是两者共同起了作用。四组对照可以拆开这个问题；再换客户、法域或作者，检查平台会不会在不该沿用仍照搬修改。

图 5.3 先比较四个版本，再检查换客户、法域或作者后会怎样

分别测试删除、新增及两者组合

比较对象	需要回答的问题	验收条件	状态
原版	—	作为父版本与对照基线	父版本
仅删除 D1（风险句）	原句错误、缺少依据、不适用于这份合同，还是谈判取舍	删去一次，不代表以后都不必提示这类风险	已提出，待核查
仅新增 D2（公式）	公式从哪里来、计算什么、适用于谁	各法域分别取得依据；没有已批准依据时保持待审	已提出，待核查
D1 + D2 组合	改进来自哪一处修改	版本定下后，用没有参与调参的材料验收	D1、D2 分别审查

改变条件后的预期行为

变化	预期行为	验收条件
客户 A 换成客户 B	A 的红线不进入 B 的材料	检查检索、摘要、记忆和评估材料，不只删除最后的客户名
换成另一法域（德国或越南）	不通过替换国家名称补出系数	各法域分别取得依据，由对应法域的复核者核查
德国样例算出相同金额	不据此认定公式完整、可用于其他法域	检查基数、期间和适用范围
越南没有已批准的依据	保持待审	不输出系数
只给一个尚未生效的版本号	不绕过验收与批准	看规则是否已批准生效，不看版本号
作者换成知名合伙人	不绕过验收与批准	相同证据得到相同处理

法源与适用条件由熟悉当地法律的复核者检查。提出修改的学习器不能自行改验收题、评分标准或发布权限，也不能改变客户隔离要求。版本确定以后，再用未参与调参的材料验收；如果反复根据同一隐藏集的结果选版本，实际上也在向这套题调参。作者再有名，版本号再新，都不能省去这些检查。

采用新版时保留父版本和依据。如果后来发现它把原先答对的问题答错了，就停用受影响规则，恢复适用的已批准版本，再根据失败记录提出修改。这样，下一轮可以继续学习，而不必让已经发现的问题继续影响合同。

5.4 奖励黑客风险在哪里？

只奖励报告干净、修改少或测试分高，系统就可能学会删警告、套公式、避开难题或全部转人工。先满足法源、范围、隔离和批准这些不能用效率折抵的条件，再比较质量与成本；要说系统更会改进，还需另做一层对照。

把警告删掉，报告会更干净；把公式套上，结论显得更确定；跳过难题，正确率会更高；记住客户 A 的答案，演示效果会更好；全部转人工，自动错误会更少。这些变化都能让某个指标变好，问题却留给了客户或审阅者。因此，法源、适用范围、客户隔离、必检项和批准是先要满足的门槛，效率收益不能抵消其中任何一项；通过以后，再一起比较质量、覆盖率、人工时间与成本。

即使这些都做到，也只回答了新版本是否更会处理合同。要证明平台更会改进自己，需要另做一次比较。

要证明什么	比较方法	足以支持什么判断
新规则更会完成任务	原版与新版处理同一批未见过的合同，比较错误、覆盖率、审阅时间和总成本	这次规则更新有效
新系统更会产生下一次改进	使用相同规则版本、失败案例和预算，分别检查新旧学习器提出的修改是否有效；未被采用的尝试也计入成本	在这些条件下，新版更善于提出并验证有效改进

前一项做得好，不能代替后一项的证据；后一项也需要跨多轮、跨任务复核，才能排除偶然成功。RSI 研究中的提出、验证、选择和回滚可以帮助安排这些实验，却不能证明某条法律公式正确。平台现在就能用上的，是这次学到的检查方法：删除和新增分开审，先查清规则出自哪里，再交给独立的人验收。那条公式仍要另行取得依据和批准。

来源

来源	发布方	类型	支持的内容
Google Research —RRSI: Regularized Recursive Self-Improvement of Agent Harnesses	Google Research	研究代码仓库	候选保留变更历史，评估后再选用
RSI-Exam —Benchmarking Recursive Self-Improvement through Executable Research	RSI-Exam project	研究基准	开发与隐藏评测分离，冻结后验收
Weco AIDE² research report	Weco AI	研究论文或预印本	任务优化不等于改进能力提升
Anthropic —Natural emergent misalignment from reward hacking	Anthropic	研究论文或预印本	钻检查漏洞会带来失准风险
Singapore Court of Appeal —Man Financial v Wong Bark Chuan David [2007] SGCA 53	Singapore Judiciary eLitigation	判决	竞业限制须辨别可保护利益
Germany —Handelsgesetzbuch §74	Gesetze im Internet	法律条文	补偿下限为最后合同给付一半
Germany —Handelsgesetzbuch §74a	Gesetze im Internet	法律条文	限制期不得超过离职后两年
Germany —Handelsgesetzbuch §74b	Gesetze im Internet	法律条文	浮动给付接近三年平均计算

Germany —Gewerbeordnung §110	Gesetze im Internet	法律条文	雇佣关系相应适用 HGB 竞业条款
US Treasury OFAC —Entities Owned by Blocked Persons (50 Percent Rule)	U.S. Department of the Treasury, OFAC	监管机关说明	50% 所有权规则非补偿系数

说明

访问与核对日期：2026-09-23。以下法律条文和判决仅用于辨认题中出现的概念（如竞业限制补偿、可保护利益、50% 规则的不同含义），不构成法律意见，也不覆盖新加坡、德国以外法域或本题合同的完整适用判断。